

LegalEase Project Blueprint

Source: docs/blueprint/project-blueprint.md + appendix

Table of Contents

- LegalEase Full-Stack Project Blueprint
- 1) System Overview
- 2) Architecture Map
- 3) Backend Deep Dive
 - 3.1 Router topology
 - 3.2 Startup and middleware
 - 3.3 Domain/service modules
 - 3.4 Data and schema layers
- 4) Frontend Deep Dive
 - 4.1 App shell and route graph
 - 4.2 Core providers and hooks
 - 4.3 Client contract layer
- 5) Runtime Workflow Modeling
 - 5.1 Auth bootstrap workflow
 - 5.2 Chat workflow (all modes)
 - 5.3 Document upload and indexing workflow
 - 5.4 Matter lifecycle workflow
 - 5.5 Learning/tuning workflow
- 6) Security and Access Control Matrix
- 7) Observability and Reliability Surfaces
- 8) Frameworks, Libraries, and Tools
- Backend
- Frontend
- Tooling and ops
- 9) API and Data Contract Highlights
- 10) Testing and Validation Surfaces
- 11) Feature Flags and Environment Controls
- 12) Small-Feature Coverage Notes
- 13) Maintenance Notes
- Feature Inventory and Code Mapping
 - A) Platform Entry and Composition
 - B) Chat and Session Features
 - C) KB, Upload, and Index Features
 - D) Matter Workspace Features
 - E) Auth, Security, and Platform Controls
 - F) Learning, Tuning, and Automation
 - G) Speech Features
 - H) Business Domains (SaaS Modules)
 - I) Frontend UX Helpers and Utility Features
 - J) Observability, Runbooks, and Ops
 - K) Test Mapping

LegalEase Full-Stack Project Blueprint

Generated on 2026-05-28.

1) System Overview

LegalEase is a full-stack legal operations platform with:

- A Next.js App Router frontend in web/ for chat, case workspaces, document operations, billing, CRM, discovery, premium tools, and settings.
- A FastAPI backend in backend/app/ exposing /api/v1/* domain routers.
- Hybrid AI architecture combining local LLM inference (Ollama / LM Studio) with cloud grounded web research (Gemini) under strict scope boundaries.
- Multi-domain persistence on SQLite (default), with optional Postgres/Redis pathways.
- Matter-scoped RAG indexes with explicit isolation controls for matter vs global KB usage.

Primary execution entry points:

- Backend app bootstrap: backend/app/main.py
- Backend route composition: backend/app/api/v1/router.py
- Frontend authenticated shell: web/app/(app)/layout.tsx
- Frontend main chat surface: web/app/(app)/page.tsx
- Frontend API contract layer: web/lib/api.ts

2) Architecture Map

flowchart LR

```
U[User Browser] --> W[Next.js UI web/app]
W -->|HTTP JSON + SSE| A[FastAPI backend/app/main.py]
A --> R[Domain Routers backend/app/api/v1/endpoints]
R --> S[Services backend/app/services]
S --> C[Core Modules backend/app/core]
C --> D[(SQLite/Postgres)]
C --> F[(FAISS Indexes)]
C --> O[Ollama/LM Studio]
C --> G[Gemini APIs]
C --> Q[(Redis Optional)]
```

Layer responsibilities:

- **Client/UI**: route-level orchestration, stream rendering, optimistic UI and status surfaces.
- **API**: auth, validation, rate limits, scoped access checks, endpoint contracts.
- **Service orchestration**: chat mode routing, hybrid fusion orchestration, speech and KB workflows.
- **Core domain/data**: matter policy, repositories, schema migrations, indexing, learning, observability.
- **State/index engines**: RAG vector indexes, relational persistence, queue/scheduler workers.

3) Backend Deep Dive

3.1 Router topology

backend/app/api/v1/router.py mounts:

- health, chat, engines, documents, kb, sessions
- premium, learning, memory, matters
- templates, drafting, clauses
- billing, trust, crm, portal, esign
- ediscovery, research, practice, dashboard, speech

3.2 Startup and middleware

From backend/app/main.py:

- Loads .env roots and optional GPU profile.
- Registers:
 - MemoryEfficiencyMiddleware (backend/app/middleware/memory_guard.py)
 - RateLimitMiddleware (backend/app/middleware/rate_limit.py)
 - CORSMiddleware with configurable origin controls (backend/app/core/config.py).
- Starts non-blocking startup thread:
 - schema ensures/migrations
 - embedding preload
 - optional RAG warmup
 - optional reindex scheduler and coach scheduler.

3.3 Domain/service modules

- **Chat + mode routing**
 - backend/app/api/v1/endpoints/chat.py
 - backend/app/services/chat_service.py
 - backend/app/services/mode_router.py
 - backend/app/services/hybrid_orchestrator.py
- **Documents + indexing**
 - backend/app/api/v1/endpoints/documents.py
 - backend/app/core/index_jobs.py
 - backend/app/core/reindex_scheduler.py
 - backend/app/core/matter_index.py
 - rag.py, kb_pipeline.py
- **Matters + lifecycle + access**
 - backend/app/api/v1/endpoints/matters.py
 - backend/app/core/matter_repo.py
 - backend/app/core/matter_policy.py
 - backend/app/core/matter_workflow.py
- **Learning + tuning**
 - backend/app/api/v1/endpoints/learning.py
 - backend/app/core/adaptive_learning.py
 - backend/app/core/learning_engine.py
 - backend/app/core/neural_finetuning.py
 - backend/app/core/llm_finetuning.py
 - backend/app/core/improvement_automation.py
 - backend/app/core/gemini_ollama_coach.py
- **Practice ops modules**
 - billing: backend/app/api/v1/endpoints/billing.py

- CRM: backend/app/api/v1/endpoints/crm.py
- trust: backend/app/api/v1/endpoints/trust.py
- e-discovery: backend/app/api/v1/endpoints/ediscovery.py
- portal/e-sign/templates/clauses/drafting endpoints in same folder.

3.4 Data and schema layers

- Operational DB access and URL selection:
 - backend/app/core/database.py
 - backend/app/core/db.py
- Schema migration and sanity:
 - backend/app/core/schema_migrations.py
 - backend/app/core/practice_schema.py
 - backend/app/core/saas_schema.py
- Major table families:
 - auth/users and logs
 - chat history + thread attachments
 - matters, hearings, tasks, deadlines, entities, evidence, members, audit
 - learning interactions/feedback/signals/promotions
 - billing + invoices, CRM leads, trust accounts/transactions
 - discovery batches/items/jobs, research logs, templates/clauses

4) Frontend Deep Dive

4.1 App shell and route graph

Top-level route containers:

- Root app provider shell: `web/app/layout.tsx`
- Authenticated shell: `web/app/(app)/layout.tsx`
- Matter scoped nested shell: `web/app/(app)/matters/[matterId]/layout.tsx`

Feature pages:

- Chat: `web/app/(app)/page.tsx`
- Documents: `web/app/(app)/documents/page.tsx`
- Matters list + create + per-tab views under `web/app/(app)/matters/`
- Billing, Intake, Discovery, Drafting, Premium, Tools, Analytics, Settings, Dashboard
- Public token flows: `web/app/portal/[token]/page.tsx`

4.2 Core providers and hooks

- Auth/session provider: `web/components/providers/AuthProvider.tsx`
- API connectivity provider: `web/components/providers/ApiConnectionProvider.tsx`
- Chat session context: `web/components/providers/ChatSessionProvider.tsx`
- Chat orchestration hook: `web/hooks/useChat.ts`
- Speech orchestration hook: `web/hooks/useSpeechToText.ts`
- Matter notifications hook: `web/hooks/useMatterNotifications.ts`

4.3 Client contract layer

`web/lib/api.ts` is the frontend backend-contract surface:

- Auth, health, chat history/threads, stream chat.
- Document upload/index/health/job polling.
- Matter CRUD, notes, timeline/hearings/tasks/deadlines, entities/evidence, audit, members, export.
- Learning feedback/signals/tuning/automation endpoints.
- Billing, CRM, trust, discovery, portal, e-sign, premium, speech.

It also includes:

- timeout/retry wrappers (`fetchWithTimeout`, `fetchWithRetry`)
- browser/server API base normalization
- robust parsing for structured backend errors
- SSE-like stream parser for `/api/v1/chat/stream`.

5) Runtime Workflow Modeling

5.1 Auth bootstrap workflow

1. User submits credentials (/api/v1/auth/login).
2. Backend validates via legacy_saas/legalease_auth.py and signs token with legacy_saas/auth_tokens.py.
3. Frontend stores token (legalease_token) and hydrates AuthProvider.
4. Protected endpoints validate bearer token via backend/app/core/auth.py.

Guardrails:

- Password policy checks in register flow.
- Live membership fetched from DB during auth dependency resolution.

5.2 Chat workflow (all modes)

sequenceDiagram

```
participant UI as Chat UI
participant API as /api/v1/chat/stream
participant CS as chat_service
participant MR as mode_router
participant KB as RAG/KB
participant WEB as Web/Gemini

UI->>API: POST stream request (mode, lang, history, matter_id)
API->>CS: validated scope request
CS->>MR: resolve mode path
alt knowledge_base
    MR->>KB: retrieval + synthesis (local)
else web_search/open_law
    MR->>WEB: grounded web answer
else deep_case/hybrid
    MR->>KB: KB evidence
    MR->>WEB: web evidence
    MR->>CS: fused response
end
end
CS-->>API: token + meta events
API-->>UI: SSE chunks + completion meta
```

Failure paths:

- Network drop: frontend abort/retry-safe handling.
- Unauthorized scope: backend rejects invalid matter scope.
- Low-confidence retrieval: not_found behavior + escalation paths.

5.3 Document upload and indexing workflow

1. UI sends file to /api/v1/documents/upload or /api/v1/matters/{id}/documents/upload.
2. Backend extracts text (PDF/OCR path), stores doc metadata.
3. Index pipeline creates chunks and embeddings.
4. index_status transitions (processing -> ready or queued/failed).
5. Job status observable via /api/v1/documents/jobs/{job_id} and dashboard health endpoints.

Controls:

- scoped indexing directories (unlinked/global vs matter-specific).
- async index job runner via backend/app/core/index_jobs.py.
- reindex scheduler via backend/app/core/reindex_scheduler.py.

- observability events in backend/app/core/observability.py.

5.4 Matter lifecycle workflow

1. Create matter (POST /api/v1/matters).
2. Attach/upload matter documents, notes, tasks, hearings, evidence.
3. Execute scoped chat and matter intelligence.
4. Archive by default on delete endpoint; hard delete available via explicit query flag.
5. Restore archived matter via restore endpoint.

Authorization:

- centralized matter context resolution in backend/app/core/matter_policy.py.
- role-aware write controls and strict scope feature flags.

5.5 Learning/tuning workflow

1. Chat interactions + signals recorded per mode/scope.
2. Feedback endpoints update adaptive stores and learning stats.
3. Neural and LLM tuning endpoints expose collect/train/status.
4. Admin scope promotion endpoint allows controlled matter->global learning promotion.
5. Automation/coach scheduler performs periodic training actions where enabled.

Safety:

- promotion endpoint rate-limited and admin-gated.
- scope keys preserve global vs matter learning boundaries.

6) Security and Access Control Matrix

- Authentication: bearer JWT style token, password hashing with bcrypt.
- Authorization:
 - global auth dependency for protected routers.
 - matter-level access context and role checks.
 - owner-only controls for sensitive operations (delete/restore/export variants).
- API abuse protection: endpoint-specific rate limiting middleware.
- Scope isolation:
 - chat scope normalization and validation.
 - matter strict enforcement flags.
- CORS restrictions configurable via environment variables.

7) Observability and Reliability Surfaces

- Structured events: backend/app/core/observability.py.
- Health endpoints:
 - liveness/readiness/public/system checks under /api/v1/health/*
 - indexing health and KB health endpoints.
- Startup snapshot utilities for operational state visibility.
- Runbooks:
 - docs/runbooks/matter-kb-ops.md
 - docs/runbooks/production-dry-run-checklist.md
 - RUNBOOK.md

8) Frameworks, Libraries, and Tools

Backend

- FastAPI, uvicorn, pydantic
- sqlalchemy, sqlite3, optional psycopg2
- redis integration for queue/session options
- faiss-cpu, sentence-transformers, transformers, torch
- OCR/PDF stack: PyPDF2, pdfplumber, PyMuPDF, easyocr, pytesseract, opencv-python-headless
- Optional ML tuning stack: datasets, accelerate, peft, trl

Frontend

- Next.js 15, React 19, TypeScript
- tailwindcss, react-markdown, remark-gfm

Tooling and ops

- pytest with marker-based suite partitioning.
- PowerShell scripts for backend/web startup and test execution.
- Docker compose deployment stack with API/web/worker/postgres/redis/nginx.
- CI workflow in [.github/workflows/ci.yml](#).

9) API and Data Contract Highlights

- Chat stream event envelope includes token, status, meta, and error event types.
- Matter dashboard contract includes matter metadata, document list (with index and privilege markers), timelines/tasks/deadlines, KB health, autopilot, smoke checks.
- Document indexing contracts expose async job IDs for long-running operations.
- Learning contracts expose mode stats, quality signals, coach status, neural training state, progress checkpoints.

10) Testing and Validation Surfaces

Main suites under tests/ include:

- RAG/KB retrieval and pipeline tests.
- API flow, chat, and session tests.
- Matter hardening, policy flag behavior, and E2E matter lifecycle tests.
- Learning, tuning, and automation tests.
- SaaS phase regressions across billing/CRM/discovery/premium.

Specific hardening-era tests:

- tests/test_matter_hardenig_regression.py
- tests/test_api_matter_e2e_flow.py
- tests/test_matter_policy_flags.py

11) Feature Flags and Environment Controls

Representative controls (see `.env.example`):

- `MATTER_STRICT_SCOPE_ENFORCEMENT`
- `MATTER_STRICT_ROLE_WRITE`
- `LEARNING_SCOPE_PROMOTION_ENABLED`
- `LEGALEEASE_MINIMAL_STARTUP`
- `LEGALEEASE_SKIP_RAG_WARMUP`
- `LEGALEEASE_EMERGENCY_STARTUP`
- indexing and scheduler related toggles
- tuning and automation toggles

12) Small-Feature Coverage Notes

The appendix ([docs/blueprint/appendix/feature-inventory.md](#)) maps detailed small features, edge helpers, and utility controls to code locations, including:

- chat escalation rules
- thread attachments
- speech fallback paths and mic diagnostics
- KB health fallback object contracts
- timeline suggestion approve/reject loops
- matter notifications polling
- per-endpoint rate-limit overrides
- legacy alias endpoints for backward compatibility

13) Maintenance Notes

When updating this blueprint:

1. Update route inventory when `backend/app/api/v1/router.py` changes.
2. Update client API coverage when `web/lib/api.ts` changes.
3. Sync matter policy section after edits in `backend/app/core/matter_policy.py`.
4. Regenerate `project-blueprint.pdf` from this Markdown.
5. Keep appendix coverage aligned with newly added minor features.

Feature Inventory and Code Mapping

This appendix catalogs major and small exposed features, mapped to implementation anchors.

A) Platform Entry and Composition

- Backend API bootstrap: `backend/app/main.py`
- V1 router composition: `backend/app/api/v1/router.py`
- Frontend root shell: `web/app/layout.tsx`
- Frontend authenticated shell: `web/app/(app)/layout.tsx`
- Frontend chat page entry: `web/app/(app)/page.tsx`

B) Chat and Session Features

- Chat submit + stream transport: web/hooks/useChat.ts, web/lib/api.ts
- Stream parser (data: events): web/lib/api.ts
- Regenerate/feedback bindings: web/components/chat/ChatViewport.tsx, web/components/chat/MessageFeedback.tsx
- Follow-up chips: web/components/chat/SuggestionPills.tsx
- Thread history and restore: backend/app/api/v1/endpoints/sessions.py, backend/app/core/chat_persistence.py
- Thread attachment upload/remove/read: backend/app/api/v1/endpoints/sessions.py, backend/app/core/thread_attachments.py
- Mode switch learning signal emission: web/app/(app)/page.tsx, web/lib/api.ts
- Chat export report: backend/app/api/v1/endpoints/chat.py, web/lib/api.ts

C) KB, Upload, and Index Features

- Document upload endpoint: backend/app/api/v1/endpoints/documents.py
- OCR extraction routes: backend/app/main.py, backend/app/api/v1/endpoints/documents.py, ocr_engine.py
- Index job async runner: backend/app/core/index_jobs.py
- Reindex automation scheduler: backend/app/core/reindex_scheduler.py
- KB health snapshot endpoints: backend/app/api/v1/endpoints/documents.py, backend/app/main.py
- KB smoke tests: backend/app/core/matter_intelligence.py, backend/app/api/v1/endpoints/documents.py
- Per-scope FAISS pathing: backend/app/core/matter_index.py
- RAG pipeline and chunk batching: rag.py, kb_pipeline.py, kb_preprocess.py

Small features:

- job polling helper in frontend: waitForIndexJob in web/lib/api.ts
- fallback health contract object: EMPTY_KB_HEALTH in web/lib/api.ts
- index status transitions (processing/ready/queued/failed) in documents endpoint.

D) Matter Workspace Features

- Matter CRUD + lifecycle endpoints: backend/app/api/v1/endpoints/matters.py
- Matter repository ops: backend/app/core/matter_repo.py
- Access context and role policy: backend/app/core/matter_policy.py
- Matter dashboard aggregation: backend/app/core/matter_workflow.py
- Matter dashboard UI: web/components/matters/MatterDashboard.tsx
- Matter settings form: web/components/matters/MatterSettingsForm.tsx
- Matter deletion/archive flows: web/components/matters/MatterDeleteModal.tsx, backend matters endpoint/repo
- Matter uploads and doc metadata flags: web/components/matters/MatterDocumentUpload.tsx, backend/app/api/v1/endpoints/matters.py
- Timeline suggestions + moderation: web/components/matters/TimelineSuggestions.tsx, matters endpoints
- Notifications polling: web/hooks/useMatterNotifications.ts

Small features:

- privileged document marker and index_status surface on dashboard.
- scoped ?open in main chat? handoff in matter layout.
- contradiction panel and witness profile helpers in matter tabs.

E) Auth, Security, and Platform Controls

- Auth endpoints (login/register/me): backend/app/main.py
- Bearer user dependency: backend/app/core/auth.py
- Token issuance/validation: legacy_saas/auth_tokens.py
- Membership source-of-truth: legacy_saas/legalease_auth.py
- Rate limits: backend/app/middleware/rate_limit.py
- Memory pressure guard headers: backend/app/middleware/memory_guard.py
- CORS config: backend/app/core/config.py

Small features:

- endpoint-specific promotion limit rule in rate-limit middleware.
- compatibility aliases for legacy /api/* routes in backend/app/main.py.

F) Learning, Tuning, and Automation

- Learning endpoints: backend/app/api/v1/endpoints/learning.py
- Adaptive interaction/feedback stores: backend/app/core/adaptive_learning.py
- Learning signal handling: backend/app/core/learning_signals.py
- Learning engine status and memory: backend/app/core/learning_engine.py
- Neural train orchestration: backend/app/core/neural_finetuning.py
- LLM fine-tuning support: backend/app/core/llm_finetuning.py
- Coach directives/scheduler: backend/app/core/gemini_ollama_coach.py, backend/app/core/coach_scheduler.py
- Improvement automation pipeline: backend/app/core/improvement_automation.py

Small features:

- scope-keyed feedback separation (global vs matter:<id>).
- admin-only scope promotion endpoint with feature gate.
- run-now and status APIs for tuning pipeline observability.

G) Speech Features

- Speech transcribe API surface: `backend/app/api/v1/endpoints/speech.py`
- Speech service implementation: `backend/app/services/speech_service.py`
- UI speech panel and recorder control: `web/components/speech/SpeechPanel.tsx`
- Voice-aware text input: `web/components/ui/VoiceTextarea.tsx`
- speech hook state machine: `web/hooks/useSpeechToText.ts`
- language code mapping: `web/lib/speechLang.ts`
- speech UI status derivation helper: `web/lib/speech/uiFlags.ts`

Small features:

- explicit browser fallback error class and handling path.
- mic device persistence and diagnostics logs.

H) Business Domains (SaaS Modules)

- Billing: backend/app/api/v1/endpoints/billing.py, web/app/(app)/billing/page.tsx
- CRM/Intake: backend/app/api/v1/endpoints/crm.py, web/app/(app)/intake/page.tsx
- Trust ledger: backend/app/api/v1/endpoints/trust.py
- E-discovery: backend/app/api/v1/endpoints/ediscovery.py, backend/app/core/job_queue.py, run_ediscovery_worker.ps1
- Research log: backend/app/api/v1/endpoints/research_log.py
- Templates/clauses/drafting: corresponding endpoints and web/app/(app)/drafting/page.tsx
- Portal and e-sign: backend/app/api/v1/endpoints/portal.py, backend/app/api/v1/endpoints/esign.py
- Premium tools: backend/app/api/v1/endpoints/premium.py, web/app/(app)/premium/page.tsx

I) Frontend UX Helpers and Utility Features

- API connection quality polling: `web/components/providers/ApiConnectionProvider.tsx`
- Chat cache persistence helpers: `web/lib/chatStorage.ts`
- Display label normalization: `web/lib/displayLabels.ts`
- Mode pills membership gating: `web/components/chat/ModePills.tsx`
- Engine and scope health strips: `web/components/chat/EngineStatusBar.tsx`, `web/components/chat/KbScopeHealth.tsx`

Small features:

- localhost to 127.0.0.1 normalization resilience in API layer.
- default retry/backoff logic shared by all client API calls.

J) Observability, Runbooks, and Ops

- Structured event emitter: `backend/app/core/observability.py`
- Startup snapshot status: `backend/app/core/startup_state.py`
- Production runbooks:
 - `docs/runbooks/matter-kb-ops.md`
 - `docs/runbooks/production-dry-run-checklist.md`
 - `RUNBOOK.md`

K) Test Mapping

- Matter hardening regressions: tests/test_matter_hardening_regression.py
- Matter lifecycle E2E: tests/test_api_matter_e2e_flow.py
- Matter policy feature flags: tests/test_matter_policy_flags.py
- Chat/session/API suites: tests/test_api_*.py, tests/test_chat_*.py
- KB/RAG suites: tests/test_kb_*.py, tests/test_rag.py
- Learning/tuning suites: tests/test_learning_*.py, tests/test_neural_finetuning.py
- SaaS phase suites: tests/test_phase*_*.py